

OP 성적서 발행기 — 개발 설명서

이 문서는 OP 성적서 발행기 애플리케이션의 아키텍처, 기술 스택, 핵심 로직 및 빌드/배포 방법에 대해 설명합니다. 본 프로젝트는 V3 마이그레이션을 통해 완전한 **독립 실행형 (Standalone)** 로컬 웹 애플리케이션으로 전환되었습니다.

[!NOTE] 사용자(비개발자) 대상 안내는 [사용설명서.md](#) 를 참고하세요.

1. 시스템 아키텍처 및 기술 스택

1-1. 기술 스택

- **Backend:** Java 17, Spring Boot 3.2.5
- **Frontend:** React 18.3.1, Vite 5.2.11
- **Data Storage:** Local JSON File (Jackson ObjectMapper)
- **PDF Generation:**
 - `poi-tl` (Word `.docx` 템플릿 데이터 치환)
 - `documents4j-local` + VBScript (MS Word COM 자동화를 통한 `.docx` → `.pdf` 변환)
 - `pdfbox` (다중 PDF 병합)
- **Packaging:** JDK 내장 `jpackage` (JRE 포함 단일 앱 이미지 빌드)

1-2. V3 마이그레이션 아키텍처 주요 변경 사항

기존 V2 아키텍처(사용자 PC에 MySQL 및 Java 별도 설치 필요)의 배포/사용 진입장벽을 해소하기 위해 다음과 같이 개선되었습니다.

1. **데이터베이스 독립:** MySQL(JPA) 의존성을 완전히 제거하고, `data/history.json` 파일에 발급 이력을 저장/조회하는 파일 기반 저장소(`JsonDataStore`)로 마이그레이션.
2. **단일 서버 통합:** 프론트엔드 React 빌드 결과물을 Spring Boot의 `src/main/resources/static` 으로 복사하여 단일 8080 포트로 서비스(`WebConfig` 설정).
3. **JRE 내장 단독 실행(Standalone) 패키징:** `jpackage` 를 사용해 실행 대상 PC에 Java(JRE/JDK) 설치 없이 즉시 구동 가능한 `.exe` 앱 이미지로 배포 구조 변경.
4. **완전한 포터블(Portable) 폴더 구조:** 실행 파일과 설정, 템플릿 파일이 하나의 폴더에 통합되어 USB 등으로 자유롭게 이동 가능.

5. 초극악 환경 방어 시스템 (Ultra-Extreme Defense):

- **자가 복구(Self-Healing)**: 정전/에러로 JSON DB가 오염될 경우 스스로 백업하고 재기동하여 서비스 중단 방지.
- **광클/매크로 방어**: 초당 다중 요청 시 `ReentrantLock` 을 통해 즉시 차단(429)하여 시스템 마비 방지.
- **OS I/O 방어**: 디스크 용량 0바이트, MAX_PATH 초과 등에 대해 친절한 에러 메시지 (Safe-Fail) 제공.

2. 개발 환경 구성

2-1. 요구 사항

- **JDK**: Java 17 이상 (설치 후 환경변수 `PATH` 및 `JAVA_HOME` 등록 필수)
- **Node.js & npm**: v18 이상 권장 (프론트엔드 빌드용)
- **Microsoft Word**: 로컬 PC 설치 필수 (개발 및 실행 시 모두 필요 - `documents4j` 의존)

2-2. 로컬 개발 실행 방법

프론트엔드(React)와 백엔드(Spring Boot)를 분리하여 실행합니다.

프론트엔드 실행

```
cd frontend
npm install
npm run dev    # 포트: 5173 (설정: vite.config.js)
```

백엔드 실행

```
cd backend
mvnw spring-boot:run    # 포트: 8080 (설정: application.yml)
```

프론트엔드에서 `/api/*` 요청 시 Vite Proxy를 통해 `localhost:8080` 으로 자동 전달됩니다.

3. 핵심 비즈니스 로직 설명

3-1. Word 템플릿 치환 (DocxTemplateService)

- 사용자가 직접 수정 가능한 외부의 `certificate_template.docx` 파일을 읽어들이습니다. (`application.yml` 의 `app.template.path` 참조)
- `poi-tl` 라이브러리를 활용하여 `{cno}`, `{sno}`, `{cedate}`, `{pdate}`, `{edate}` 등의 변수 태그를 입력된 값으로 치환합니다.
- **후처리 로직:**
 - 치환 후 원본 템플릿에 남아있는 글자색(주황/파랑 등) 오류를 방지하기 위해, 치환된 모든 텍스트(Paragraph/Run)의 색상을 검정(`000000`)으로 강제 변경합니다.
 - 표의 모든 셀 내용을 상하/좌우 중앙 정렬 처리합니다.

3-2. MS Word PDF 변환 (MsWordPdfConverter)

- `documents4j` 라이브러리는 백그라운드에서 MS Word 프로세스를 구동해 `.docx` 를 `.pdf` 로 변환합니다.
- **버그 우회(Fallback) 처리:** 일부 환경에서 `documents4j` 기본 스크립트 실행이 차단되거나 실패하는 현상을 방지하기 위해, 제공된 커스텀 `docx2pdf.vbs` 스크립트를 `cscript.exe` 로 직접 호출하여 변환 안정성을 극대화합니다.
- 생성 방식(개별 ZIP vs 통합 PDF)에 따라, 여러 개의 `.pdf` 바이트 배열이 반환되면 `PDFMergerUtility` (Apache PDFBox)를 이용해 1개의 파일로 병합합니다.

3-3. 발급 번호 자동 채번 규칙

- **형식:** `OP` + 발행일(YYYYMMDD) + 순번(4자리 숫자) (예: `OP202606050001`)
- **채번 로직:** 저장된 JSON 데이터(`history.json`)를 읽어, 동일한 발행일 접두사 (`OP20260605`)를 가진 데이터 중 가장 마지막 번호를 찾은 뒤 `+1` 하여 할당합니다.
- 사용자 입력 `Start No` 가 주어질 경우 해당 번호부터 강제로 채번합니다.

3-4. 자동 브라우저 런처 및 Graceful Shutdown (BrowserLauncher)

- Spring Boot 구동 완료 시(`ApplicationReadyEvent`), Chrome 또는 Edge 브라우저를 **App Mode**(`--app`) 로 실행하여 주소창 없는 깔끔한 윈도우 UI를 제공합니다. (`600x540` 크기 지정)
- **독립 프로파일 사용:** `--user-data-dir` 옵션으로 임시 프로ファイルを 사용하여, 기존에 켜져 있는 Chrome과 세션을 분리하고 프로세스 충돌을 방지합니다.
- **Graceful Shutdown:** 브라우저 창이 닫히면 데몬 스레드가 이를 감지하고 `applicationContext.close()` 를 호출합니다. Spring Boot의 `server.shutdown: graceful` 설정에 따라 진행 중인 PDF 생성 등의 작업이 끝날 때까지 최대 30초 대기 후 안전하게 백엔드 서버를 종료합니다.

4. 데이터 저장 및 관리 메커니즘 (JsonDataStore)

MySQL/JPA 의존성을 제거하고 구현된 파일 기반 스토리지의 특징은 다음과 같습니다.

- **단일 진입점:** 모든 데이터 접근은 `JsonDataStore` 클래스를 통합니다.
- **동시성 제어 (TOCTOU 방지):** `ReentrantReadWriteLock` 의 쓰기 잠금 상태에서 파일을 읽고, 수정하고, 다시 쓰는 과정을 하나의 `update()` 메서드로 묶어 **원자적(Atomic) 연산**을 보장합니다. 다중 발급 요청 시 데이터 덮어쓰기로 인한 유실을 완벽히 방지합니다.
- **저장 방식:** 새로운 데이터가 추가되거나 변경되면, 변경된 전체 List를 JSON 파일에 즉시 Overwrite 합니다.

5. 원클릭 빌드 및 배포 프로세스

단일 실행형 배포판을 만들기 위해 루트 경로에 `build.ps1` 스크립트가 준비되어 있습니다.

5-1. build.ps1 실행 흐름

1. **프론트엔드 빌드:** `frontend/` 디렉토리에서 `npm run build` 를 실행합니다.
2. **정적 리소스 병합:** 생성된 `dist/` 내 파일들을 `backend/src/main/resources/static/` 폴더로 덮어씁니다.
3. **Fat JAR 빌드:** `backend/` 디렉토리에서 Maven을 사용해 실행 가능한 하나의 `.jar` 파일을 생성합니다. (`mvnw clean package -DskipTests`)
4. **App Image 패키징:** `jpackage` 명령어를 사용하여 JRE를 내장한 독립 실행 폴더(앱 이미지)를 `dist/` 하위에 생성합니다.
5. **외부 파일 복사 및 정리:** 앱 실행 폴더 내부에 `certificate_template.docx` 및 `docx2pdf.vbs` 파일을 복사하고, 불필요한 `.ico` 파일을 제거합니다.

5-2. 배포 디렉토리 구조 (dist/)

빌드가 완료되면 생성되는 구조는 다음과 같습니다. `OP_Certificate_Generator` 폴더 전체를 ZIP으로 압축하여 배포하며, 이 폴더는 어디로든 이동 가능합니다.

```
dist/
├── OP_Certificate_Generator/
│   ├── OP_Certificate_Generator.exe    # 실제 실행 파일
│   ├── runtime/                       # 내장된 경량 JRE
│   ├── app/                           # Spring Boot Fat JAR 및 설정
│   └── certificate_template.docx       # 외부 Word 템플릿 (수정 가능)
└── # 완전한 독립 포터블 폴더
```

└─ docx2pdf.vbs

└─ data/

PDF 변환 VBScript
구동 시 발급 이력 JSON 파일이 생성되는 폴더

6. 주요 트러블슈팅 및 개선 기록

이슈	해결 및 개선
V3: 사용자 설치 제약 (MySQL, Java)	jpackage Standalone 앱 이미지 생성 + JsonDataStore 기반 DB 무설치 환경 달성
동시성 데이터 유실 버그 (BUG-1)	JsonDataStore.update() 구현으로 읽기-수정-쓰기 로직을 원자적(Atomic) 처리
UX 에러 메시지 잔류 (BUG-2)	폼 재제출 시 즉각적인 에러 상태 초기화 로직 프론트엔드에 추가
서버 강제 종료 위험 (WARN-5)	System.exit(0) 제거, Graceful Shutdown 도입 및 브라우저 데몬 스레드 연동
Chrome 기존 프로세스 충돌	--user-data-dir 옵션을 통해 독립적인 임시 프로필을 생성하여 Failed to fetch 에러 근본 해결
앱 윈도우 크기 최적화	CSS 폼 크기(max-width: 450px)에 맞춰 Chrome 윈도우 사이즈를 600x540 콤팩트하게 고정
포터블 디렉토리 구조 통합	템플릿 경로를 ../ 에서 ./ 로 변경하여 OP_Certificate_Generator 폴더 단일 이동 보장